

14 장 학습 과제

과제 1. **pair**와 구조체 비교

상품 ID와 수량을 반환하는 함수를 `std::pair<int, int>`로 작성한다. 같은 함수를 이름 있는 `ItemCountResult` 구조체로 다시 작성한다.

구조적 바인딩을 이용해 두 반환값을 받고, 호출부에서 각 값의 의미를 파악하기 쉬운 쪽을 설명한다.

과제 2. **tuple** 분해와 **apply**

이름, 레벨, 체력, 이동 속도를 저장하는 튜플을 만든다. `std::get()`, 구조적 바인딩, `std::apply()`를 각각 사용해 모든 원소를 출력한다.

구조적 바인딩을 값, 참조, 상수 참조로 작성하고 원본 수정 여부와 복사 발생 가능성을 비교한다.

과제 3. **optional** 검색 결과

학생 이름 벡터에서 지정한 이름의 인덱스를 찾는 함수를 작성한다. 찾으면

`std::optional<std::size_t>`에 인덱스를 저장하고 찾지 못하면 `std::nullopt`를 반환한다.

`operator*`, `value()`, `value_or()`를 사용한 호출 코드를 각각 작성하고 값이 없을 때의 차이를 설명한다.

과제 4. **optional**과 포인터

`std::vector<Player>`에서 ID로 플레이어를 찾는 함수를 포인터 반환형과

`std::optional<Player>` 반환형으로 각각 작성한다.

원본 객체 수정, 복사 비용, 값 부재, 컨테이너 재할당 후 유효성 측면에서 두 인터페이스를 비교한다.

과제 5. **variant** 설정 값

정수, 실수, 문자열을 저장할 수 있는 `SettingValue`를 `std::variant`로 정의한다. `std::get_if()`로 각 값을 출력하는 함수와 `std::visit()`으로 출력하는 함수를 작성한다.

지원하지 않는 대안 타입을 하나 추가한 뒤 방문자 코드에서 누락이 어떻게 발견되는지 확인한다.

과제 6. **variant** 상태 모델

캐릭터 상태를 `Idle`, `Moving`, `Attacking`, `Dead` 구조체로 나누고 `std::variant`로 묶는다. 각 상태에 필요한 데이터만 저장한다.

`std::visit()`으로 상태별 갱신 함수를 작성하고 여러 `bool` 멤버로 상태를 표현한 구조와 유효하지 않은 상태 가능성을 비교한다.

과제 7. **any** 속성 저장소

문자열 키와 `std::any` 값을 저장하는 속성 테이블을 작성한다. 정수, 실수, 문자열 값을 등록하고 포인터 형식 `std::any_cast()`로 안전하게 읽는다.

잘못된 타입을 요청했을 때 값 형식과 포인터 형식의 차이를 확인한다.

`std::unique_ptr<int>`를 직접 저장할 수 없는 이유도 설명한다.

과제 8. **reference_wrapper** 파티 구성

서로 독립적으로 생성된 **Player** 객체 세 개를 **std::vector<std::reference_wrapper<Player>>**에 저장한다. 컨테이너 순회로 모든 플레이어의 체력을 변경한다.
std::vector<Player*>와 비교해 **nullptr**, 문법, 소유권, 수명 측면의 차이를 정리한다.

과제 9. **span** 배열 처리

정수 원소를 두 배로 만드는 함수를 **std::span<int>**로 작성한다. C 배열, **std::array**, **std::vector**를 각각 전달한다.
std::span<const int>를 받는 합계 함수도 작성하고 **const std::span<int>**와 원소 수정 가능 여부를 비교한다.

과제 10. **span** 부분 구간과 무효화

정수 벡터에서 **first()**, **last()**, **subspan()**으로 부분 범위를 만든다. 부분 **span**을 통해 원본 원소를 수정한다.
벡터 재할당 전후의 **data()** 주소를 출력하고 기존 **span**을 재할당 후 사용할 수 없는 이유를 반복자 무효화와 연결해 설명한다.

과제 11. **string_view** 분리 함수

key=value 형식 문자열을 두 개의 **std::string_view**로 나누는 함수를 작성한다. 구분자가 없을 때의 반환 규칙도 정한다.
문자열 리터럴, 지역 **std::string**, 임시 **std::string**을 전달했을 때 반환된 뷰의 수명을 분석한다.

과제 12. 널 종료 확인

긴 **std::string**의 앞부분 다섯 문자만 가리키는 **std::string_view**를 만든다. 스트림에 뷰 자체를 출력한 결과와 **data()**를 C 문자열로 출력한 결과를 비교한다.
C API에 안전하게 전달하기 위해 **std::string**으로 변환하는 코드를 작성한다.

과제 13. **bitset** 권한 관리

읽기, 쓰기, 실행 권한을 **std::bitset<3>**으로 표현한다. 각 권한 설정, 해제, 반전, 검사 함수를 작성한다.
같은 권한을 **enum class** 비트 플래그로도 구현하고 타입 표현과 출력 편의성을 비교한다.

과제 14. **C++20** 비트 함수

여러 **std::uint32_t** 값에 **std::popcount()**, **std::has_single_bit()**, **std::bit_floor()**, **std::bit_ceil()**, **std::bit_width()**를 적용한다.
입력값 0, 1, 63, 64, 65의 결과를 표로 정리하고 각 함수의 용도를 설명한다.

과제 15. **bit_cast**와 수치 변환

float 값을 **static_cast<std::uint32_t>()**와 **std::bit_cast<std::uint32_t>()**로 변환한다. 두 결과가 다른 이유를 수치값과 객체 표현의 차이로 설명한다.
bit_cast 대상 타입의 크기와 **trivially copyable** 조건을 **static_assert**로 확인한다.

과제 16. 엔디언 직렬화

std::uint32_t 값을 항상 빅 엔디언 네 바이트로 저장하는 함수를 작성한다. 시프트와 마스크를 사용하고 호스트 엔디언에 의존하지 않게 한다.
저장한 네 바이트를 다시 정수로 복원하는 함수도 작성한다.

과제 17. 의미 타입

플레이어 ID, 아이템 ID, 수량을 모두 `int`로 받는 함수를 작성한 뒤 각 값을 별도 래퍼 타입으로 변경한다.

인수 순서를 잘못 전달했을 때 원시 정수 버전과 의미 타입 버전의 컴파일 결과를 비교한다.

과제 18. 유효한 상태 설계

네트워크 요청 결과를 성공, 시간 초과, 서버 오류로 나누고 `std::variant`로 표현한다. 각 상태에 필요한 데이터만 포함한다.

불리언 플래그와 선택적 문자열을 사용하는 단일 구조체 버전도 작성하고 만들어질 수 있는 모순 상태를 열거한다.

과제 19. 데이터 표현 선택

작업별로 `pair`, 구조체, `optional`, `variant`, `any`, `reference_wrapper`, `span`, `string_view` 중 적합한 타입을 선택하고 근거를 작성한다.

- 두 좌표 반환
- 사용자 검색 실패
- 정수·실수·문자열 설정
- 확장 모듈의 임의 속성
- 기존 객체 목록 참조
- 연속 배열 일부 전달
- 문자열 일부 읽기
- 성공값과 오류 코드 반환

소유권, 타입 집합의 고정 여부, 수명, 오류 정보, 복사 비용을 판단 기준에 포함한다.

과제 20. 댕글링 분석

`std::reference_wrapper`, `std::span`, `std::string_view`가 댕글링 상태가 되는 코드를 각각 작성하되 무효 객체를 실제로 역참조하거나 순회하지 않는다.

대상 객체 소멸, 컨테이너 재할당, 임시 객체 소멸 중 어떤 원인으로 무효화되는지 주석으로 기록하고 안전한 인터페이스로 수정한다.

참고. C++23의 `std::expected`

`std::optional<T>`는 값이 없는 상태를 표현하지만 실패 이유를 저장하지 않는다.

`std::expected<T, E>`는 성공값 `T` 또는 오류값 `E` 중 하나를 저장한다.

```
#include <expected>
#include <string>
#include <string_view>

enum class ParseError
{
    Empty,
    InvalidCharacter,
    OutOfRange
};

std::expected<int, ParseError> ParsePositiveInt(std::string_view text);
```

성공하면 값에 접근한다.

```
auto result = ParsePositiveInt("120");

if (result)
{
    std::cout << *result << '\n';
}
```

실패하면 `error()`로 오류값을 얻는다.

```
if (!result)
{
    ParseError error = result.error();
}
```

오류 결과는 `std::unexpected`로 만든다.

```
std::expected<int, ParseError> ParsePositiveInt(std::string_view text)
{
    if (text.empty())
    {
        return std::unexpected(ParseError::Empty);
    }

    int value = 0;

    for (char character : text)
    {
        if (character < '0' || character > '9')
        {
            return std::unexpected(ParseError::InvalidCharacter);
        }

        int digit = character - '0';

        if (value > (std::numeric_limits<int>::max() - digit) / 10)
        {
            return std::unexpected(ParseError::OutOfRange);
        }

        value = value * 10 + digit;
    }
}
```

```

    return value;
}

```

value()는 성공값을 반환하고 실패 상태이면 `std::bad_expected_access<E>`를 발생시킨다.

```

try
{
    int value = result.value();
}
catch (const std::bad_expected_access<ParseError>& exception)
{
    ParseError error = exception.error();
}

```

반환할 성공 데이터가 없고 성공 여부와 오류만 필요하면 `std::expected<void, E>`를 사용할 수 있다.

```

std::expected<void, SaveError> SaveSettings(const Settings& settings)
{
    if (!CanOpenFile())
    {
        return std::unexpected(SaveError::OpenFailed);
    }

    WriteSettings(settings);
    return {};
}

auto result = SaveSettings(settings);

if (!result)
{
    HandleSaveError(result.error());
}

```

C++23의 `expected`는 성공과 실패 흐름을 연결하는 연산도 제공한다.

```

auto result = LoadText("config.txt")
    .and_then(ParseConfig)
    .transform(BuildSettings)
    .or_else(LogLoadError);

```

`and_then()`: 성공값으로 `expected`를 반환하는 함수를 호출

```
transform(): 성공값을 다른 값으로 변환  
or_else(): 오류 상태를 처리하거나 다른 expected로 변환  
transform_error(): 오류값의 타입이나 내용을 변환
```

연결된 각 연산은 앞 단계가 성공했을 때만 성공 경로를 실행한다. 오류가 발생하면 남은 성공 연산을 건너뛰고 오류 상태를 전달한다.

expected는 예외를 자동으로 대체하지 않는다. 오류가 함수의 정상적인 결과에 가깝고 호출자가 현 위치에서 검사하거나 전달할 때 적합하다. 메모리 부족이나 내부 불변식 위반처럼 일반적인 반환 흐름으로 처리하기 어려운 실패까지 모두 **expected**로 바꾸면 호출부의 검사 코드가 과도해질 수 있다.

C++23 모드와 <expected>를 지원하는 표준 라이브러리가 필요하다.

